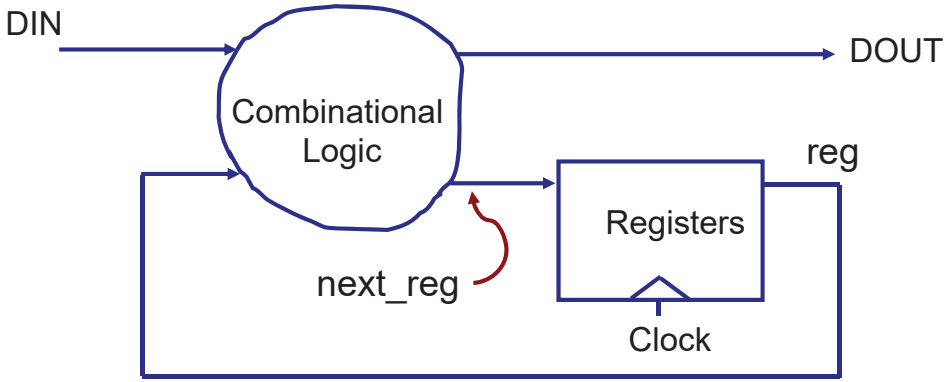
EE332
Digital System Design

Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

Chapter 6: Modeling at the RT Level

- A register transfer level (RTL) design consists of a set of registers connected by combinational logic.



6.1 Combinational circuit

- A combinational circuit, by definition, is a circuit whose output, after the initial transient period, is a function of current input.
- It has no internal state and therefore is “memoryless” about the past event (or past inputs).

```
signal A, B, Cin, Cout : bit;
…
process (A, B, Cin) is
begin
   Cout <= (A and B) or ((A xor B) and Cin);
end;
```

To describe a combinational circuit

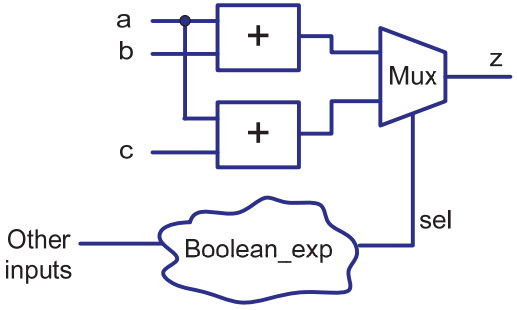
- The variables or signals in the process must not have initial values.
- A signal or a variable must be assigned a value before being referenced.
- The arithmetic operators (such as +, -, *, etc), relational operators (such as <, >, =, etc), and logic operators (such as and, or, not, etc) can be used in an expression.

Operator sharing

when a control signal is 1, z <= a + b;
Otherwise, z <= a + c;

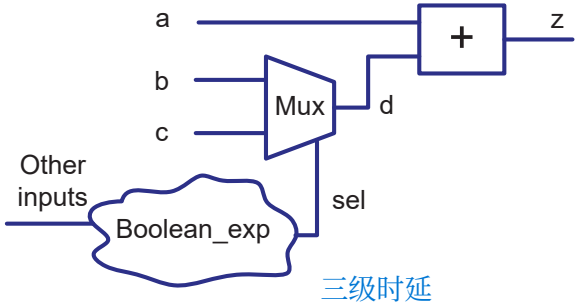
- One way to reduce the overall size of synthesized hardware is to identify the resources that can be used by different operations. This is know as *resource sharing*.

```
sel <= c1 xor c2;
z <= a + b when sel='1' else
     a + c;
```



两级时延
1. **加法器** or boolean
2. 选择器

```
sel <= c1 xor c2;
d <= b when sel='1' else
     c;
z <= a + d;
```



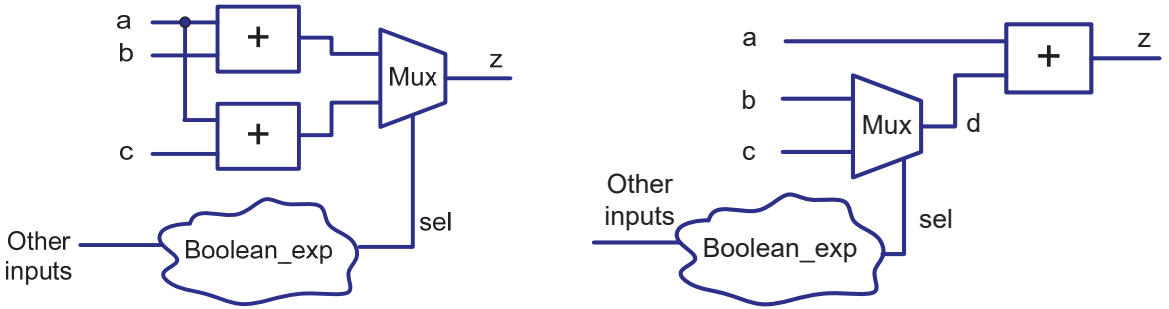
三级时延
1. boolean
2. 选择器
3. **加法器**

- Performing resource sharing normally introduces some overhead and may penalize performance.
  - In the above examples, assume T_{adder}, T_{mux}, $T_{boolean}$,
    - For the circuit not sharing the adders:

$$T = \max (T_{adder}, T_{boolean}) + T_{mux}$$

    - For the circuit sharing the adders:

$$T = T_{adder} + T_{boolean} + T_{mux}$$



Shaping the circuit

- Using VHDL code, it is possible to outline the general shape of the circuit.

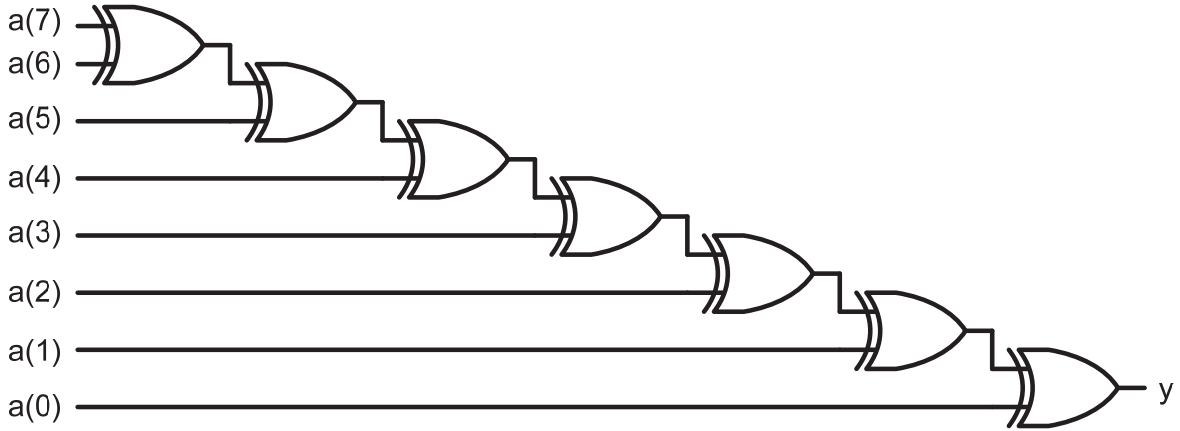
Reduced-xor circuit

$$y = a_7 \oplus a_6 \oplus a_5 \oplus a_4 \oplus a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

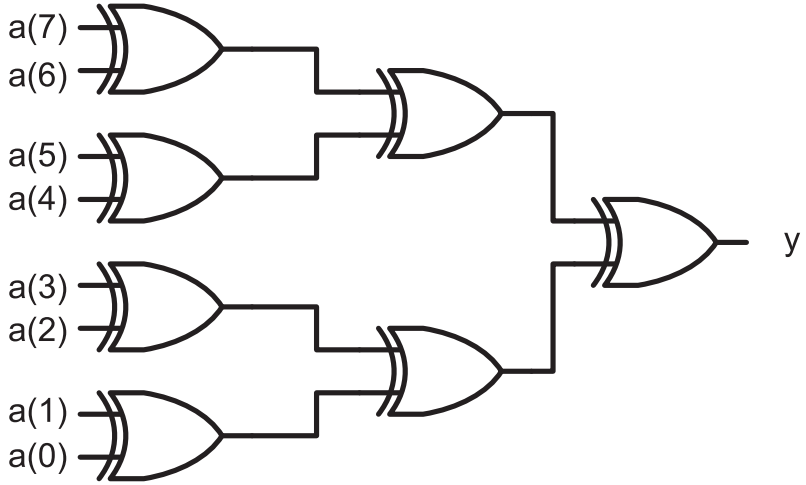
```
signal a: std_logic_vector (7 downto 0);
signal y: std_logic;
…
y <= a(7) xor a(6) xor a(5) xor a(4) xor a(3)
        xor a(2) xor a(1) xor a(0);
```

从左到右执行

$y <= (((((((a_7 \oplus a_6) \oplus a_5) \oplus a_4) \oplus a_3) \oplus a_2) \oplus a_1) \oplus a_0);$



$$y <= ((a_7 \oplus a_6) \oplus (a_5 \oplus a_4)) \oplus ((a_3 \oplus a_2) \oplus (a_1 \oplus a_0));$$



Example:
Combinational adder-based multiplier

					a_4	a_3	a_2	a_1	a_0	multiplicand
x					b_4	b_3	b_2	b_1	b_0	multiplier
					a_4b_0	a_3b_0	a_2b_0	a_1b_0	a_0b_0	
				a_4b_1	a_3b_1	a_2b_1	a_1b_1	a_0b_1		
			a_4b_2	a_3b_2	a_2b_2	a_1b_2	a_0b_2			
+		a_4b_3	a_3b_3	a_2b_3	a_1b_3	a_0b_3				
	a_4b_4	a_3b_4	a_2b_4	a_1b_4	a_0b_4					
y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0	product

The algorithm includes three tasks:

- Multiply the digits of the multiplier (b4, b3, b2, b1 and b0) by the multiplicand A = (a4, a3, a2, a1, a0) one at a time to obtain b4*A, b3*A, b2*A, b1*A and b0*A.

  $$bi * A = (a4 \cdot bi, a3 \cdot bi, a2 \cdot bi, a1 \cdot bi, a0 \cdot bi)$$

- Shift bi * A to left by i position.
- Add the shifted bi * A terms to obtain the final product.

						a_4 b_4	a_3 b_3	a_2 b_2	a_1 b_1	a_0 b_0
x										
					a_4b_1	a_4b_0 a_3b_1	a_3b_0 a_2b_1	a_2b_0 a_1b_1	a_1b_0 a_0b_1	a_0b_0
				a_4b_2	a_3b_2	a_2b_2	a_1b_2	a_0b_2		
+			a_4b_3	a_3b_3	a_2b_3	a_1b_3	a_0b_3			
		a_4b_4	a_3b_4	a_2b_4	a_1b_4	a_0b_4				
	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0

Initial description of an adder-based multiplier

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity mult5 is
port (a, b : in std_logic_vector(4 downto 0);
        y: out std_logic_vector(9 downto 0));
end entity mult5;

architecture comb1_arch of mult5 is
constant WIDTH : integer := 5;
signal au, bv0, bv1, bv2, bv3, bv4: std_logic_vector(WIDTH–1 downto 0);
signal p0, p1, p2, p3, p4, prod: std_logic_vector(2*WIDTH–1 downto 0);
```

x					a_4 b_4	a_3 b_3	a_2 b_2	a_1 b_1	a_0 b_0
+		a_4b_4	a_4b_3 a_3b_4	a_4b_2 a_3b_3 a_2b_4	a_4b_1 a_3b_2 a_2b_3 a_1b_4	a_4b_0 a_3b_1 a_2b_2 a_1b_3 a_0b_4	a_3b_0 a_2b_1 a_1b_2 a_0b_3	a_2b_0 a_1b_1 a_0b_2	a_1b_0 a_0b_1	a_0b_0
	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0

使用中间信号来寄存输入输出，避开输入只能放在右边输出只能放在左边的限制

每一位都是b(0)

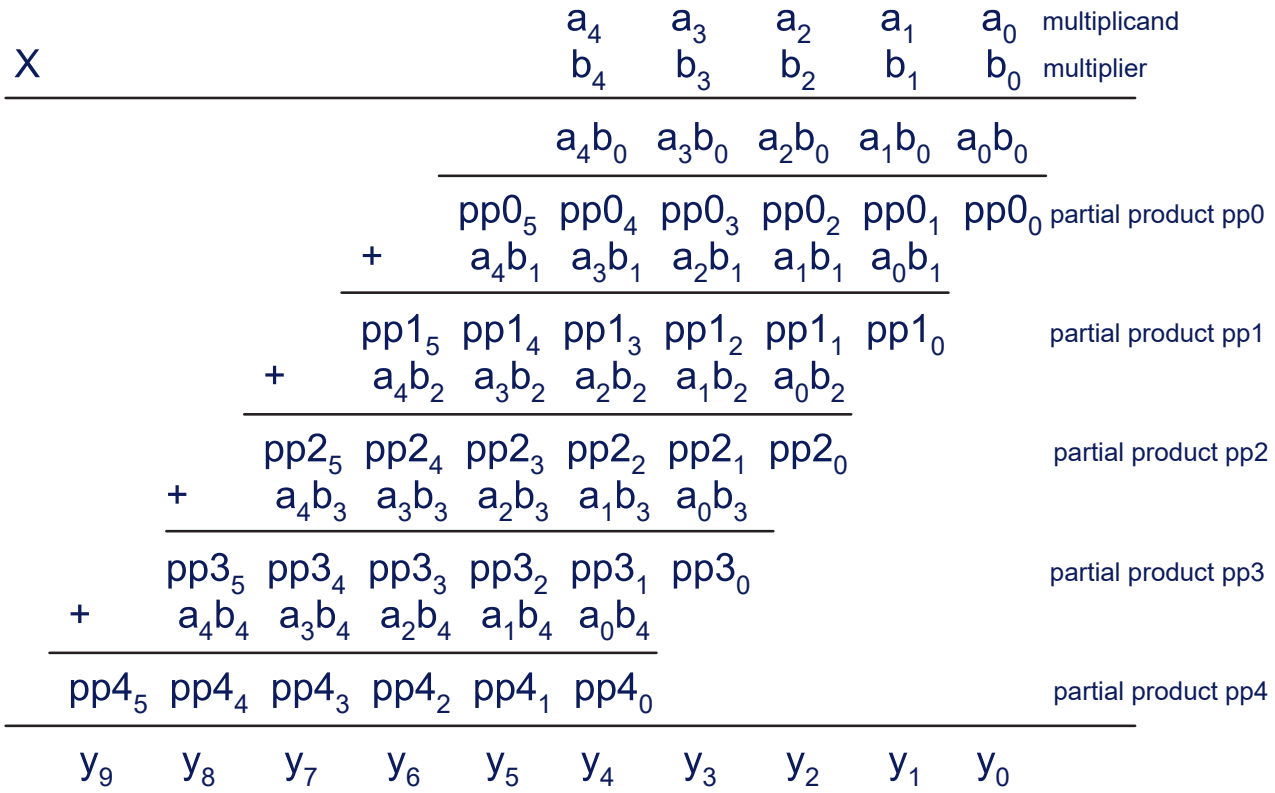
```
begin
    au <= a;
    bv0 <= (others => b(0));
    bv1 <= (others => b(1));
    bv2 <= (others => b(2));
    bv3 <= (others => b(3));
    bv4 <= (others => b(4));
    p0 <= "00000" & (bv0 and au);
    p1 <= "0000" & (bv1 and au) & '0';
    p2 <= "000" & (bv2 and au) & "00";
    p3 <= "00" & (bv3 and au) & "000";
    p4 <= '0' & (bv4 and au) & "0000";
    prod <= ((p0+p1)+(p2+p3))+p4;
    y <= prod;
end architecture comb1_arch;
```

Array aggregate:
a VHDL construct to assign a value to an object of array data type.

same
```
v <= "1011";
v <= ('1', '0', '1', '1');
v <= (3=>'1', 2=>'0', 1=>'1', 0=>'1');
v <= (3|1|0=>'1', 2=>'0');
v <= (2=>'0', others=>'1');
v <= (others=>'0');
```

第3位为1，第2位为0，第1位为1，第0位为1

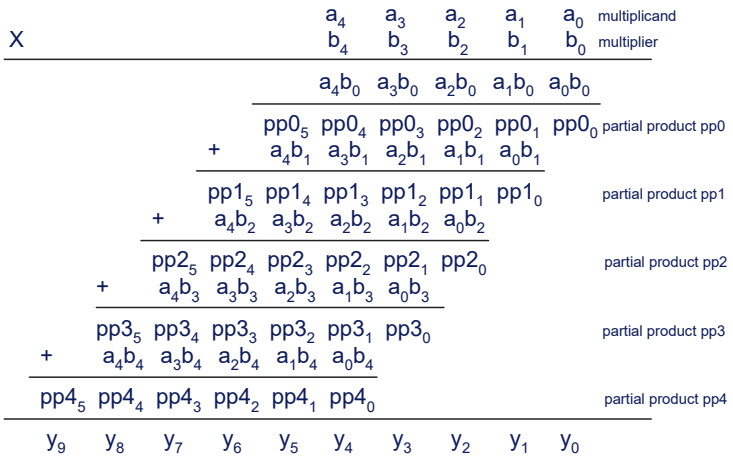
x					a_4 b_4	a_3 b_3	a_2 b_2	a_1 b_1	a_0 b_0	
					a_4b_0	a_3b_0	a_2b_0	a_1b_0	a_0b_0	
				a_4b_1	a_3b_1	a_2b_1	a_1b_1	a_0b_1		
			a_4b_2	a_3b_2	a_2b_2	a_1b_2	a_0b_2			
+		a_4b_3	a_3b_3	a_2b_3	a_1b_3	a_0b_3				
	a_4b_4	a_3b_4	a_2b_4	a_1b_4	a_0b_4					
	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0



改进：将十位加法器优化成六位加法器

More efficient description of an adder-based multiplier

```
architecture comb2_arch of mult5 is
constant WIDTH : integer := 5;
signal au, bv0, bv1, bv2, bv3, bv4: std_logic_vector (WIDTH–1
    downto 0);
signal pp0, pp1, pp2, pp3, pp4: std_logic_vector (WIDTH downto 0);
signal prod: std_logic_vector (2*WIDTH–1 downto 0);
```




```
begin
    au <= a;
    bv0 <= (others => b(0));
    bv1 <= (others => b(1));
    bv2 <= (others => b(2));
    bv3 <= (others => b(3));
    bv4 <= (others => b(4));
    pp0 <= '0' & (bv0 and au);
    pp1 <= ('0' & pp0(WIDTH downto 1))+ ('0' & (bv1 and au));
    pp2 <= ('0' & pp1(WIDTH downto 1))+ ('0' & (bv2 and au));
    pp3 <= ('0' & pp2(WIDTH downto 1))+ ('0' & (bv3 and au));
    pp4 <= ('0' & pp3(WIDTH downto 1))+ ('0' & (bv4 and au));
    prod <= pp4 & pp3(0) & pp2(0) & pp1(0) & pp0(0);
    y <= prod;
end architecture comb2_arch;
```

缺点：时延较长

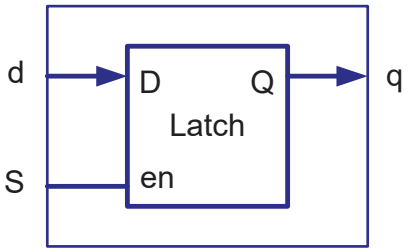
6.2 Sequential circuit

- A sequential circuit is a circuit that has an internal state, or memory.
- Its output is a function of current input as well as the internal state. Thus the output is affected by current input values as well as past input values.
- A synchronous sequential circuit, in which all memory elements are controlled by a global synchronizing signal, greatly simplifies the design process and is the most important design methodology.
- Flip-flops and latches are two commonly used one-bit memory devices.

6.2.1 Latch

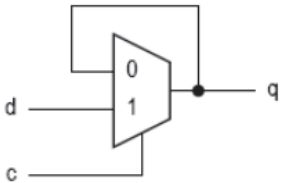
- A latch is a level-sensitive memory device.

```
signal S, d, q: bit
.....
process (S, d) is
begin
    if (S='1') then
        q <= d;
    end if;
end process;
```

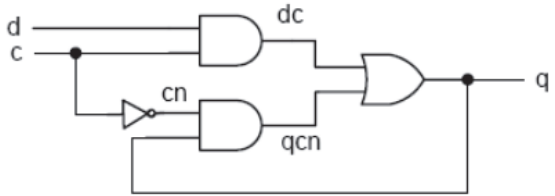


Truth table

S	q*
0	q
1	d



Conceptual diagram



Gate-level diagram

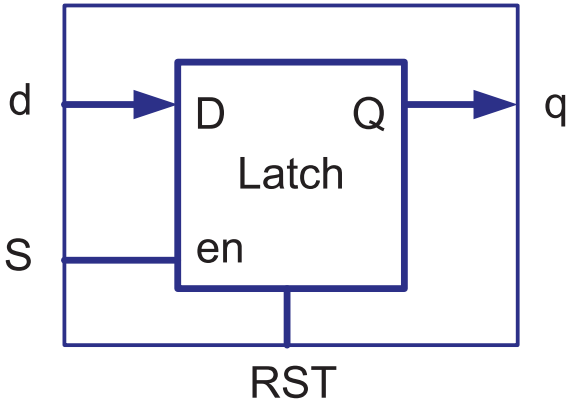
- In general, latches are synthesized from incompletely specified conditional expressions in a combinational description.
- Latch inferences occur normally with **if** statements or **case** statements.
- To avoid having a latch inferred, assign a value to the signal under all conditions.

```
signal S, d, q: bit
.....
process (S, d) is
begin
    if (S='1') then q <= d;
    else q <= '0';
    end if;
end process;
```

asynchronous reset or preset

- An asynchronous reset (or preset) will change the output of a latch to 0 (or 1) immediately.

```
signal S, RST, d, q: bit
.....
process (S, RST, d) is
begin
    if (RST = '1') then
        q <= '0';
    elsif (S='1') then
        q <= d;
    end if;
end process;
```



优先级：RST > S > d
优先级高的先出现在if statement中

6.2.2 Flip-Flops (f/f)

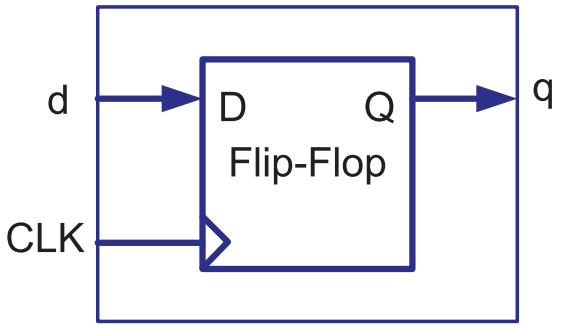
- A flip-flop is an edge-triggered memory device.
- To detect the rising edge (or falling edge), or the event occurred for a signal, we can make use of the attribute of a signal.

```
signal CLK : bit;
.....
CLK'event                    true if CLK changes its value.
CLK'event and CLK = '1'      true for the CLK rising edge
CLK'event and CLK = '0'      true for the CLK falling edge
```

- The **event** attribute on a signal is the most commonly used edge-detecting mechanism. It operates on a signal and returns a Boolean value. The result is true if the signal shows a change in value.

An example of a simple flip-flop

- An edge triggered flip-flop will be generated from a VHDL description if a signal assignment is executed on the rising (or falling) edge of another signal.

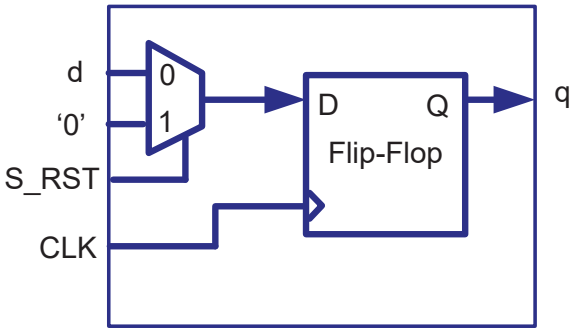



```
entity dff is
port (d, CLK: in bit; q: out bit);
end entity dff;
architecture behavior of dff is
begin
process (CLK) is
begin
    if (CLK'event and CLK='1') then
          q <= d;
    end if;
end process;
end architecture behavior;
```

和锁存器不同，**敏感列表中没有**d

Synchronous sets and resets

- Synchronous inputs set (preset) or reset (clear) the output of flip-flops when they are asserted. The assignment will only take effect while the clock edge is active.



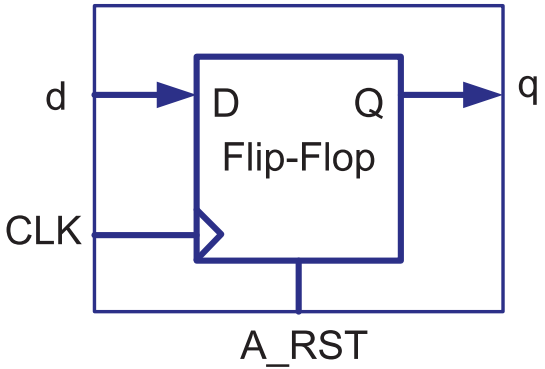
synchronous reset: S_RST不会出现在flip-flop的端口中

```
signal CLK, d, q, S_RST: bit;
.....
process (CLK) is
begin
    if (CLK'event and CLK='1') then
        if (S_RST = '1') then
            q <= '0';
        else
            q <= d;
        end if;
    end if;
end process;
```

Asynchronous sets and resets

- Asynchronous inputs set (preset) or reset (clear) the output of flip-flops whenever they are asserted independent of the clock.

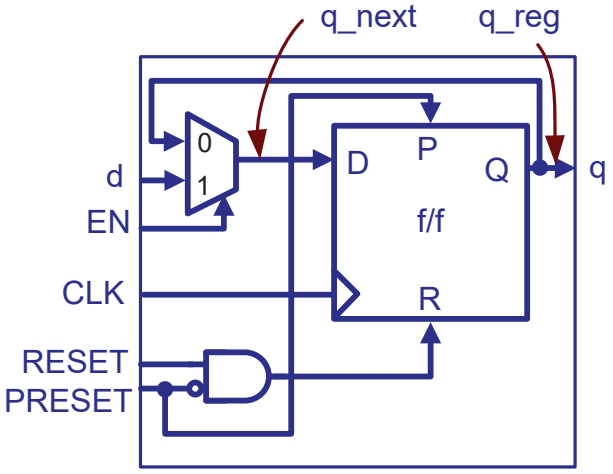
```
signal CLK , A_RST, d, q: bit;
.....
process (CLK, A_RST) is
begin
    if (A_RST = '1') then
        q <= '0';
    elsif (CLK'event and CLK='1') then
        q <= d;
    end if;
end process;
```



A f/f with more than one asynchronous input

```
signal CLK , RST, PRST, EN: bit;
signal d, q: bit;
.....
architecture two_seg of dff_en is
signal q_reg, q_next : bit;
begin
process (CLK, PRST, RST) is
begin
    if (PRST = '1') then
          q_reg <= '1';
    elsif (RST = '1') then
          q_reg <= '0';
    elsif (CLK'event and CLK='1') then
          q_reg <= q_next;
    end if;
end process;
```

process中定义 状态方程

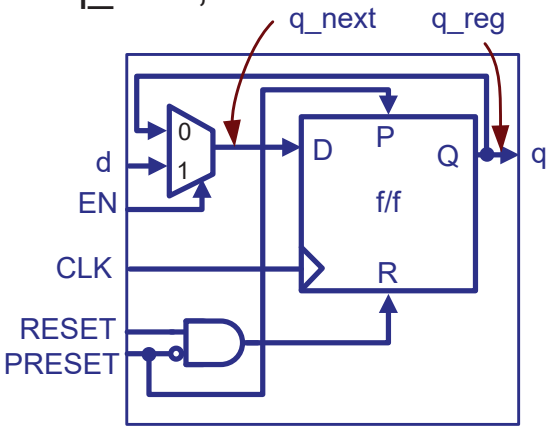


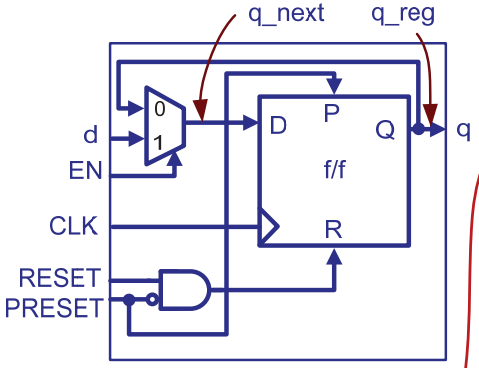
PRESET优先级高于RESET
当PRESET=1，与门输出被锁定为0

```
q_next <= d when EN = '1' else      驱动方程
          q_reg;
q <= q_reg;          输出方程
end architecture two_seg;
```

6.2.3 VHDL templates for sequential circuits

- An RTL circuit can be described in two segments:
  - A synchronous section updates the register information at the rising edge of the clock.
    - q_reg <= q_next;
  - A combinational section describes combinational logics, for example, update q_next;





A synchronous section

```
if (CLK'event AND CLK='1') then
    q_reg <= q_next;
    -- CLK is the clock input to reg q
end if;
```

or

A synchronous section with asynchronous inputs

```
if (async_sig = '1') then
    q_reg <= '0';
    -- active high asynchronous reset
elsif (CLK's event AND CLK='1') then
    q_reg <= q_next;
    -- CLK is the clock input to reg Q
end if;
```

+

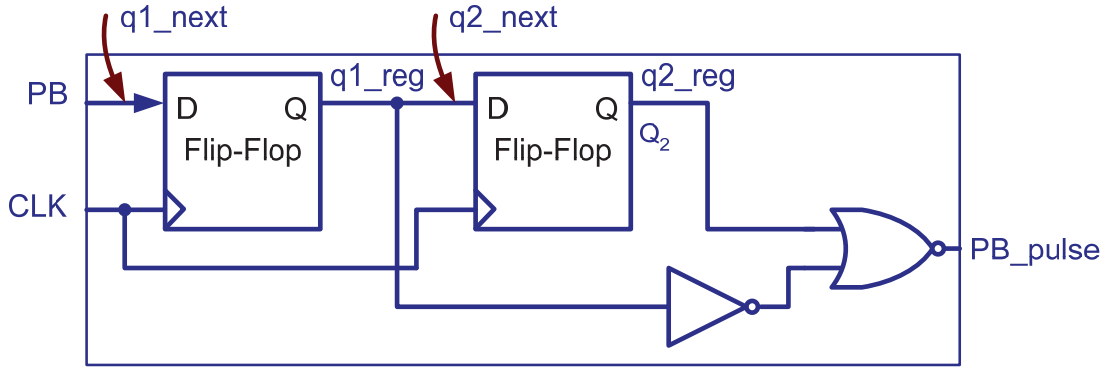
A combinational section

```
q_next <= expression;
-- other combinational logics.
```

An example

```
entity PULSER is
    port (CLK, PB : in bit ;
          PB_pulse : out bit);
end PULSER;

architecture BHV of PULSER is
    signal q1_reg, q2_reg,
           q1_next, q2_next : bit;
begin
    process (CLK) is
    begin
        if (CLK'event and CLK='1') then
            q1_reg <= q1_next;
            q2_reg <= q2_next;
        end if;
    end process;
        q1_next <= PB;
        q2_next <= q1_reg;
        PB_pulse <= (not q1_reg) nor q2_reg;
end architecture BHV;
```

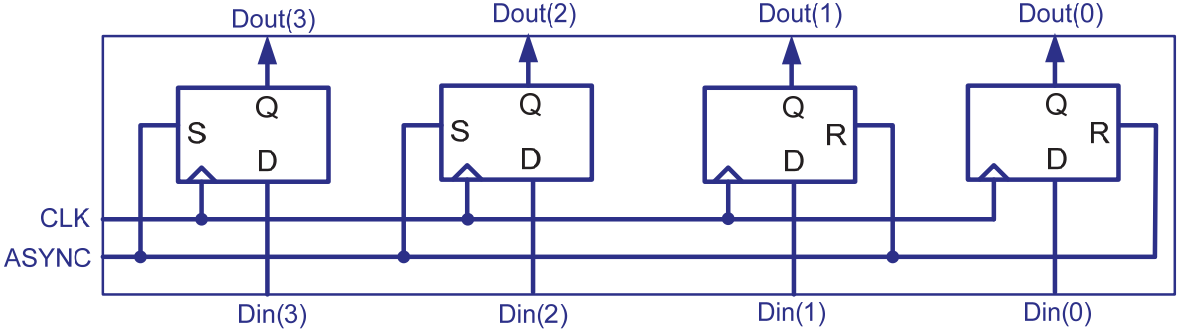


6.2.4 Registers

-- 4-bit simple register

```
signal CLK , ASYNC : bit;
singal Din, Dout :
   bit_vector (3 down to 0);
.....
```

```
process (CLK, ASYNC) is
begin
   if (ASYNC = '1') then
         Dout <= "1100";
   elsif (CLK'event and CLK='1') then
         Dout <= Din;
   end if;
end process;
```



-- 4-bit serial-in and serial-out shift register

```
signal CLK ,d, q : bit;
.....
architecture two_seg of shift_register is
signal r_reg, r_next: bit_vector (3 downto 0);
begin
process (CLK) is
begin
    if (CLK'event and CLK='1') then
        r_reg <= r_next;
    end if;
end process;
r_next <= d & r_reg(3 downto 1);
q <= r_reg(0);
end architecture two_seg;
```